

Reflection and Traceability Report on Software Engineering

Team #12, Streamliners
Mahad Ahmed
Abyan Jaigirdar
Perna Prabhu
Farhan Rahman
Ali Zia

1 Changes in Response to Feedback

This section summarizes feedback from TAs and course rubrics, instructor expectations, peer and design reviews, teammates, cross-team coordination (where applicable), the project supervisor (including post-Rev 0 alignment after the standalone MacSync pivot—see the Reflection on Project Management section for meeting frequency), and structured usability-test participants. Each item below states the *source*, the *feedback*, and the *change* (documentation, process, or code) so graders can trace rationale without opening every link.

1.1 Problem Statement and Goals

The table below summarizes the feedback and the changes made in response to it.

Table 1: Problem Statement and Goals — Feedback Response Summary

Source	Feedback Item (Brief)	Change Made (Brief)
TA Rubric	Main objective/focus unclear; integrated project context missing	Added explicit project objective, scope focus, and integration context in problem statement.
TA Rubric	Missing discussion: security/privacy, platform scope, integrations, performance	Added security/privacy concerns, mobile+web scope, third-party integration notes, and high-traffic expectations.
TA Rubric	Stakeholder hierarchy too rigid/over-structured	Replaced primary/secondary/tertiary hierarchy with a single stakeholder list reflecting overlaps.
TA Rubric	Goals not clearly measurable; missing security/privacy goal	Rewrote goals with intent + measurable criteria + rationale; added Security & Privacy goal.
TA Rubric	Reflection should be individual (Q1/Q2), team-wide (Q3)	Restructured reflection appendix: Q1/Q2 per member; Q3 team response.

1.2 Development Plan and Proof of Concept Demonstration Plan

The table below summarizes the feedback and the changes made in response to it.

Table 2: Development Plan — Feedback Response Summary

Source	Feedback Item (Brief)	Change Made (Brief)
TA Rubric (LO13)	Section 10 tech choices lacked rationale/pros/cons	Expanded “Expected Technology” with project-specific rationale, trade-offs, and limitations per tool/tech.
TA Rubric	Roles missing technical responsibilities and overlap plan	Added technical ownership (primary/secondary) across frontend, mobile, backend, DB, CI/CD.
TA Rubric	Workflow not at Contributor’s-Guide level	Added PR/review expectations, labels, quality gates, and Definition of Done.
TA Rubric	Scheduling section shallow; requested timetable-style detail	Added a high-level timetable table with phases and planned outcomes.
TA Rubric	POC plan needed more context + failed-POC consequences	Added project context, expanded risks, added scenario-based demo + failure-mode check, defined failed POC.
TA Rubric	Formatting: justified alignment not applied in places	Removed <code>flushleft</code> usage in communication plan section to avoid forced left alignment.
TA Rubric	GitHub link should point to main repo page	Updated link from board to repository URL.
TA Rubric	Reflection should be individual (Q1/Q2), team-wide (Q3)	Restructured reflection appendix: Q1/Q2 per member; Q3 team response.

1.3 SRS and Hazard Analysis

Software Requirements Specification (SRS).

Rev 1 SRS content lives in the AsciiDoc sources under `docs/SRS/` (built to `docs/SRS/index.pdf`). Table 3 summarizes feedback and edits; deliverable-level Rev 0→ Rev 1 summary also appears in Table 6.

Table 3: SRS — Feedback Response Summary

Source	Feedback (Brief)	Item	Change (Brief)	Made
TA / peer review	Some functional and NFR statements were hard to verify directly from acceptance-style tests		Clarified expected behaviour, error paths, and measurable NFR targets; aligned requirement wording with derivations in docs/VnVPlan/VnVPlan.tex and results in docs/VnVReport/VnVReport.tex.	
TA Rubric	Traceability between requirements and verification artifacts needed to be obvious for grading		Kept stable FR/NFR identifiers across SRS parts, MIS/MG, VnV Plan, and VnV Report; cross-checked module and scenario coverage against SRS sections (system goals and functional areas).	
TA / readability	System context, scope, and naming should match the Hazard Analysis boundary and design docs		Harmonized LEMS scope, integration boundaries, and component terminology with docs/HazardAnalysis/HazardAnalysis.tex and the Module Guide; reduced cross-document naming drift.	
Usability & product feedback	Refund request and organizer event-edit flows were missing or place-held in the product; requirements needed to catch up		Updated scenario and requirement coverage for end-to-end refunds and working admin event edit, consistent with Rev 1 implementation and VnV evidence.	

Traceability between SRS and Hazard Analysis (bidirectional).

The SRS specifies behaviours and qualities (FR/NFR) for LEMS/MacSync; the Hazard Analysis (`docs/HazardAnalysis/HazardAnalysis.tex`) records FMEA hazards on the same subsystems (payments, access control, sign-ups) and derives SSR-* safety and security requirements with concrete mitigations. **SRS** → **HA**: SRS-scoped workflows and data (e.g. payments, RBAC/FBAC, capacity, notifications) are the same capabilities the HA analyses for failure and abuse; HA assumptions and the system-boundary figure align with the integrations and actors described in SRS environment and system parts. **HA** → **SRS**: SSR requirements and roadmap entries in HA are reflected in SRS non-functional, security, and operational expectations that VnV tests cite alongside FR/NFR IDs (system tests in the VnV Plan often name both a functional requirement and a hazard or mitigation identifier, e.g. payment and breach scenarios). Together, the SRS + HA pair explains *what* must hold and *what can go wrong*, with VnV linking both to tests.

Hazard Analysis.

Table 4 summarizes TA feedback on the Hazard Analysis deliverable and the corresponding edits in `docs/HazardAnalysis/HazardAnalysis.tex`.

Table 4: Hazard Analysis — Feedback Response Summary

Source	Feedback Item (Brief)	Change Made (Brief)
TA (overall)	A diagram in the section defining system boundaries would help show how components interact	Added a system-boundary diagram (boxed layout: LEMS web/mobile application areas, PostgreSQL, payment and notification integration interfaces to external providers) in <code>docs/HazardAnalysis/HazardAnalysis.tex</code> under <i>System Boundaries and Components</i> , with caption and PDF-traceable figure label <code>fig:system-boundary</code> .
TA / readability	Boundary narrative should be easy to reconcile with the architecture and naming in the rest of the HA doc	Renamed boundary list wording to LEMS where inconsistent; rewrote the boundary paragraph to state in-scope components and that external payment/notification providers are treated as critical dependencies within the analysis boundary.
TA / rubric polish	Spelling, grammar, and professional presentation (supporting boundary and FMEA readability)	Removed a stray punctuation typo in the introduction workflow sentence; fixed “dependencies,” incorrect plural “API’s,” and tightened out-of-scope API wording; corrected duplicated FMEA subfigure caption (Part 2).

Note: Rubric scores for method depth, hazard completeness, and peer-review issue quality reflect process and prior drafts; the changes above address the documented TA request for a boundary diagram and alignment/readability of the boundaries section.

1.4 Design and Design Documentation

Design feedback is summarized in Table 5; the narrative below adds file-level detail and cross-team context.

Table 5: Design documentation (MIS / MG) — Feedback Response Summary

Source	Feedback (Brief)	Item	Change (Brief)	Made
Peer design review	MIS symbols and acronyms pointed readers to the SRS or MG instead of standing alone		Expanded in-document symbol/acronym table and added a per-module “sufficient to build” note in docs/Design/SoftDetailedDes/MIS.tex.	
TA / team review	Module state for payments, sign-up, and access control was too implicit for an independent implementer		Rewrote M1/M3/M5 state with explicit structures (<code>SignUpRecords</code> , versioned policy storage, payment/refund records, <code>roleAssignments</code>); moved session-timeout expectations into the MIS.	
Peer review	MG UI figures lacked orientation; abbreviations were inconsistent with MIS self-containment		Added explanatory text before figures, fixed a caption typo, and defined abbreviations inside docs/Design/SoftArchitecture/MG.tex.	
Cross-team (History Manager)	External History Manager work needed a traceable, citable integration record		Documented Team 10 contribution with public PR https://github.com/OmarHassanAdelhamid/Five-of-a-Kind-capstone-project-/pull/328 ; completed MIS appendix placeholders tied to that collaboration.	

Module Interface Specification (MIS).

In response to design-review feedback, we strengthened the MIS so it better supports an independent implementer and stands alone as a document: the symbol and acronym table was expanded and explicitly scoped to this deliverable (no deferring readers to the SRS or MG for definitions). We added a short subsection stating how each module section is intended to be sufficient to build from (interfaces, typed state, assumptions, behaviour). Payment (M1), sign-up (M3), and access control (M5) state was rewritten using explicit data structures—for example consolidated payment and refund records, a named `SignUpRecords` map, and versioned policy storage (`policyByVersion`, `currentPolicyVersion`) with correct `roleAssignments` semantics. Other edits included tightening the module hierarchy wording, stating session-timeout expectations inside the MIS instead of pointing to other docs, replacing appendix placeholder text, and documenting the History Manager work for Team 10 with a public pull request: <https://github.com/OmarHassanAdelhamid/Five-of-a-Kind-capstone-project-/pull/328>. Unused SRS cross-document label imports were removed from the MIS source.

Module Guide (MG).

We added brief narrative before each user-interface figure to explain what the screenshot shows and which modules it relates to, fixed a caption typo, and noted that abbreviations in the MG are defined in that document itself for consistency with the MIS approach.

1.5 VnV Plan and Report

VnV Report:

- **Traceability readability feedback:** FR and module traceability matrices were hard to read when scaled. **Change made:** moved matrices to become larger and wide matrices to landscape pages instead of shrinking width\resizebox, improving readability without reducing font size.
- **NFR consistency feedback:** report NFR labels did not consistently match the VnV Plan. **Change made:** aligned the report to plan IDs NFR1–NFR6 and added a dedicated NFR trace crosswalk table for one-to-one mapping.
- **Scope/rigour feedback:** earlier wording implied full equivalence to planned load/usability activities. **Change made:** explicitly documented where lighter methods were used, what was partially verified, and what remains as scope gaps (for example webhook idempotency, reliability monitoring, and audit depth).
- **Rev 0 product feedback:** “request refund not yet working” and “admin cannot edit created event.” **Change made:** implemented backend refund endpoint and frontend refund workflow, and replaced admin event-edit placeholder with a working edit form.

VnV Plan:

In response to TA feedback, we expanded the symbols and acronyms table (beyond the placeholder row), removed instructor template prompts (**wss**) and replaced them with final text, and tightened vague objectives and usability language—including concrete definitions for elevated load (tied to existing NFR concurrency cases) and measurable usability criteria (task success, timings, Likert item for NFR3). The appendix now documents symbolic parameters from those tests and states how optional usability surveys relate to NFR3. After the Module Interface Specification (MIS) was available, we filled the unit-test section: implementation verification references modules M1–M8; for each module we added representative unit test cases (**UT-M***) mapped to MIS access programs and exceptions, plus a traceability table to SRS areas. Module-level cases live under the section titled **Unit Test Description** in `docs/VnVPlan/VnVPlan.tex`. Revision history records version 1.1 (MIS-dependent content) and 1.2 (TA feedback pass).

1.6 Implementation, coding standards, and repository completeness

Code and standards.

The team followed a consistent stack-idiomatic style (e.g. TypeScript/ESLint in frontend and backend packages) with meaningful identifiers, module boundaries aligned to MIS modules M1–M8 in design reviews, and review gates for payment and auth changes. TypeScript and shared enums reduce ambiguous numeric codes where practical; reviewers are pointed to MIS access programs and exceptions as the contract for unit tests (**UT-M***).

Repository completeness (capstone template expectations).

Root-level onboarding files are present and maintained relative to the course template: `README.md`, `INSTALL.md`, `CONTRIBUTING.md`, and `LICENSE`. Per-component instructions exist under `src/frontend/README.md`, `src/backend/README.md`, and selected `docs/*/README.md` folders. Continuous integration runs automated checks (e.g. GitHub Actions) on pull requests to catch regressions before merge. This reflection does not replace line-level code review; it documents where graders should look for style, CI, and module-to-code mapping alongside the MIS and repo layout.

2 Extras

Grading note: this section is split into **Extra 1** and **Extra 2** so rubric lines map cleanly to identifiable deliverables.

Extra 1 — User documentation (Member and Staff/Admin guides)

This project completed instructor-approved extra user documentation aligned with the capstone problem-statement goals for an MES-facing event platform (member mobile experience and staff administration).

We produced two companion PDF manuals under [docs/Extras/UserManual/](#), each version 1.0.0 (April 2026):

- **MacSync Member Guide** (`User_Manual.tex`) — end-user documentation for McMaster students using the iOS/Android app: account creation and `@mcmaster.ca` verification, navigation (Events, My Tickets, Notifications, Profile), registering for free and paid events (including Stripe checkout and optional table/bus seat selection), viewing tickets and entry QR codes, requesting refunds, notification types, and profile settings with a short glossary.
- **MacSync Staff & Admin Guide** (`Admin_Manual.tex`) — documentation for the web admin portal (`admin.macsync.org`): portal overview, login and navigation, RBAC (built-in and custom roles, managing role per event, System Admin), creating/editing/deleting events and field reference, table/bus configuration, registered attendees and check-in columns, venue report PDF download, sending attendee notifications (including cooldown), user search and admin actions, warnings for destructive operations, and glossary terms (RBAC, managing role, venue report, etc.).

The manuals use consistent structure (document history, TOC, callouts for notes/warnings/tips) so MES can hand them to members and staff at rollout.

Extra 2 — Usability testing report

We documented a structured usability evaluation in [docs/Extras/UsabilityTesting/UsabilityTesting.tex](#) (dated April 6, 2026), covering both surfaces called out in scope: the React Native/Expo mobile app for student attendees and the Next.js web admin dashboard for MES organizers. The report states the plan (recruitment up to five participants per group, on-campus sessions, think-aloud, screen/audio recording with consent, tasks AT-1–AT-5 and OT-1–OT-5) and quantitative targets (e.g. task completion, errors, SUS ≥ 70).

The included results summary reports ten participants (five per group) in a first round (March 2026 per the report’s results template): mean SUS 76.5 (attendees) and 72.0 (organizers), both graded “Good,” with task completion rates and mean times tabulated per task. A usability issues log records ten findings (e.g. post-sign-up email verification clarity, bus sign-up discoverability, refund entry point labelling, admin event-form validation feedback, notification wording, role-management navigation, back-stack behaviour, analytics revenue labelling, payment error copy, loading states) with recommended fixes and Nielsen severity ratings; the conclusion notes five Severity 3 issues to address before a live

event and a planned second test round after fixes. The report also captures ethical safeguards (consent, anonymized IDs, secure storage/deletion of recordings) and an informed-consent appendix.

3 Design Iteration (LO11 (PrototypeIterate))

MacSync did not start as a single, self-contained product. Early in the capstone, the work was split across three groups, each advancing different features and assumptions in parallel. That structure helped us explore ideas quickly, but it also made end-to-end behaviour hard to guarantee: interfaces, schedules, and UX conventions could diverge, and MES stakeholders still needed one dependable system they could hand to students and volunteers for real large events. After weighing those integration risks against the society’s need for something shippable and supportable, we reset and built MacSync from scratch as a standalone application—a dedicated mobile experience for members and a web admin portal, backed by our own services, so the platform could work on its own without depending on other teams’ codebases or other teams’ release schedules. The change was driven by client-facing goals (predictable registration, payments, check-in, and staff workflows) more than by internal convenience: a standalone product meant fewer “whose repo is this?” problems and let us own quality for the full stack.

From that baseline, iteration took the form of modeling, building, and re-evaluating. We moved from informal sketches and dispersed prototypes to formal artifacts (requirements, architecture, module interface specs, and guides) so that each revision could be reviewed against MES scenarios—paid and free events, seating, notifications, roles, and refunds—rather than against fragile cross-repo glue. Implementation followed the same loop: early builds surfaced gaps (missing or placeholder flows, unclear admin paths) that we prioritized by severity and by what organizers need on event day.

Structured evaluation tightened that loop. Usability testing with McMaster attendees and MES-style organizers (documented in our extras report) showed that the product was broadly usable—both cohorts reached “Good” SUS scores—but also highlighted concrete friction: for example, confusion after sign-up about email verification, hard-to-find bus sign-up and refund entry points on mobile, and weak validation and clarity in parts of the admin dashboard (including notification wording and role-management navigation). Those findings did not merely add bug reports; they explained *why* users stalled (unclear next step, labels that did not match mental models, missing feedback after actions), which directed design and implementation work toward clearer first-run experiences, more discoverable secondary tasks, and stronger error and loading feedback for staff.

We also iterated in response to review and pilot use of the product. Section 1 notes Revision 0 gaps that mattered to MES operations—for example, members needing a working refund path and staff needing to edit events after creation

rather than living with placeholders. Addressing those items aligned the implementation with how the client actually runs formal and semi-formal events, where policies and venues change and payment exceptions happen.

In hindsight, the largest iteration was strategic—moving from a multi-group feature split to a single standalone MacSync—because it set the constraints under which all later prototypes made sense. Subsequent iterations refined that product through usability testing, feedback on real workflows, and course reviews—not only through merging code internally.

4 Design Decisions (LO12)

4.1 Project challenge level (LO12)

We classify this capstone as high challenge for a four-person, two-term software project. The work spans a member mobile client, a web admin portal, server APIs and data, third-party payments (Stripe), notifications, role-based access control across multiple MES-style organizers, and real event-day situations like registration spikes, seating logistics, refunds, and check-in. The team also produced a full documentation stack (planning, hazard analysis, design, verification, user materials) while repivoting from a multi-group split effort to a standalone product (Section 3). Together, these factors exceed a typical single-surface CRUD app in technical breadth, risk, and coordination load.

4.2 Extras (LO12)

The instructor-approved extras for this project are: **Extra 1** — user documentation (the MacSync Member Guide and the MacSync Staff & Admin Guide under `docs/Extras/UserManual/`); and **Extra 2** — usability testing (plan, metrics, tasks, and summarized results in `docs/Extras/UsabilityTesting/UsabilityTesting.tex`). Section 2 summarizes their contents; they constitute additional Rev 1 deliverables beyond the core course set.

4.3 Limitations

Several limitations shaped what we designed and how thoroughly we could validate it. Team size and time cap ambition: we prioritized MES-critical paths (event lifecycle, tickets, payments, roles, notifications, basic reporting) over nice-to-haves and deferred some non-functional depth (the VnV Report explicitly records scope gaps such as webhook idempotency, reliability monitoring, and audit depth). Dependence on third parties (Stripe, mobile push infrastructure, hosting) limits control: we integrate rather than replicate payment processing, which reduces PCI scope but ties behaviour to external APIs and policies. Single-organization pilot reality means assumptions about traffic and abuse are educated but not production-proven at massive scale. Testing economy: usability samples are small (Nielsen-style); findings are directional and were used to

prioritize UX and validation fixes rather than to claim statistical certainty.

These limitations pushed us toward clear module specs (MIS/MG) so builds did not depend on word-of-mouth, Stripe instead of home-grown payments, straightforward wording in VnV about what we did not fully test, and UX changes informed by usability sessions.

4.4 Assumptions

We assumed McMaster-authenticated membership for general users (@mcmaster.ca-based sign-up), networked clients at use time, and good-faith student organizers coordinating with MES governance. We assumed staff using the admin portal have access to a modern browser and that MES can designate System Admins sparingly for elevated actions. We assumed Stripe remains an acceptable processor for society ticket sales and that notification delivery via standard mobile channels is sufficient for event comms (with in-app history as backup). Design choices (email-bound accounts, RBAC with a managing role per event, QR check-in, PDF venue report) rest on these assumptions; breaking them would require different identity, offline, or compliance models.

4.5 Constraints

Hard course constraints included fixed deadlines, required deliverable types, TA/instructor rubrics, and reflection/trace requirements between Rev 0 and Rev 1. Institutional constraints: student IP ownership policy; no industry NDA driving hidden requirements. Technical constraints: deploying a credible demo on approved stacks (React Native/Expo, Next.js, service backend), keeping a clear line from requirements to tests in our docs, and keeping payment handling out of custom card storage. Client constraints: MES needs something volunteers can run for real events—so features like refunds and editable events are not optional polish; they drove Rev 1 implementation priorities.

4.6 How limitations, assumptions, and constraints influenced design

The combination led to a dual-surface architecture: mobile-first for attendees (discovery, purchase, ticket/QR in pocket) and web for staff (dense tables, role management, reports). RBAC with per-event managing roles balances flexibility for many clubs against a need to limit who can edit payouts, attendee data, and notifications. Stripe was chosen to bound security scope and leverage hosted checkout instead of storing payment credentials. Formal design documents offset team turnover risk and TA expectations for implementable specs. Where constraints forbade exhaustive production hardening, we documented residual risk (hazard analysis, VnV scope statements) instead of overstating coverage.

4.7 Revision 0 → Revision 1 — traceability of changes

Table 6 lists the substantive documentation, verification, and implementation changes between Rev 0 and Rev 1 so an instructor does not need to infer them from other sections. (Section 1 gives feedback-by-source detail; here the focus is a single deliverable-level change list.)

Table 6: Rev 0 → Rev 1 — explicit change traceability (LO12)

Deliverable	Rev 0 situation / gap	Rev 1 change (what we did)
Problem Statement	Objective and integrated-project context were too implicit; security/privacy, platform scope, integrations, and performance were thin; stakeholder tiers were overly rigid; goals were weakly measurable; security/privacy goal missing.	Added explicit main objective and integration context; expanded security/privacy, mobile+web scope, integrations, and high-traffic expectations; replaced rigid stakeholder hierarchy with a unified list; rewrote goals with measurable criteria and rationale; added Security & Privacy goal; reflection appendix split Q1/Q2 per member and Q3 team-wide per rubric.
Development Plan	Technology section lacked tool-specific rationale; roles lacked technical ownership; workflow needed contributor-grade detail; schedule was shallow; POC plan needed clearer failure modes/context; formatting and GitHub link issues; reflection structure.	Expanded expected technology with rationale, trade-offs, and limits; added primary/secondary technical ownership across frontend, mobile, backend, DB, CI/CD; added PR/review norms, labels, quality gates, Definition of Done; added timetable table; strengthened POC plan (context, risks, scenario demo, failed POC definition); removed forced flushleft in communication plan; pointed GitHub link to repository root; reflection appendix Q1/Q2/Q3 structure aligned with rubric.
Software Requirements (SRS)	Rev 0 draft gaps: testability of some FR/NFR statements, cross-document naming relative to HA/design, and keeping requirement IDs aligned with VnV as the product matured.	Rev 1 AsciiDoc under docs/SRS/ : clearer acceptance-style wording and measurable NFR targets; harmonized LEMS scope and integration language with HA and design; updated scenario/requirement coverage for refunds and admin event edit; feedback trace in Table 3; explicit SRS↔HA narrative in Section 1.

Continued on next page

Table 6: *Continued*

Deliverable		Rev 0 situation / gap	Rev 1 change (what we did)
Hazard Analysis		First draft; TA requested a system-boundary interaction diagram and clearer alignment/readability.	Added system-boundary figure and narrative alignment (LEMS application, database, external payment/notification dependencies); spelling/grammar polish (intro typo, out-of-scope list, FMEA part caption); traceability table in Reflect & Trace (Section 1, Table 4).
Module Interface (MIS)	Interface Spec	Early MIS draft; symbols deferred to other docs; some module state underspecified for independent build; placeholders; cross-import noise.	Version 1.1: expanded in-document symbols/acronyms; sufficiency-for-build note per module; explicit state for payment (M1), sign-up (M3), access control (M5); session-timeout expectations internalized; appendix completed; Team 10 History Manager contribution linked (https://github.com/OmarHassanAdelhamid/Five-of-a-Kind-capstone-project-/pull/328); removed unused SRS label imports; TA review items incorporated per revision history.
Module Guide (MG)	Guide	Figures lacked narrative; caption typo; abbreviations depended on other docs.	Version 1.1: explanatory text before UI figures; fixed caption typo; MG defines its own abbreviations for consistency with MIS self-containment.
VnV Plan		Symbol table placeholder-heavy; template prompts remained; load/usability targets vague; unit-test detail waited on MIS.	Versions 1.1–1.2: expanded acronyms; removed <code>wss</code> instructor placeholders; concrete elevated-load definition tied to NFR cases; measurable usability criteria (tasks, timing, Likert for NFR3); appendix parameters; implementation verification populated with <code>UT-M*</code> cases for modules M1–M8, MIS mapping, SRS trace table.
VnV Report		Trace matrices scaled poorly; NFR IDs misaligned with plan; wording overstated completeness of load/usability; product gaps on refund and admin edit.	Landscape/wider matrices (no aggressive <code>resizebox</code>); NFR labels aligned to NFR1–NFR6 with crosswalk table; scope language clarified partial methods and gaps; implemented end-to-end refund request path and working admin event edit (backend + frontend).

Continued on next page

Table 6: *Continued*

Deliverable	Rev 0 situation / gap	Rev 1 change (what we did)
Reflect & Trace	Template-only or incomplete reflection on feedback and LO coverage.	Added structured feedback tables (PS, Dev Plan, Hazard Analysis), design-doc narrative (MIS/MG), VnV bullet responses, Extras (manuals + usability report), LO11 iteration narrative, and this LO12 section with explicit Rev 0→ Rev 1 table.
Extras (beyond core set)	Not part of core Rev 0 package.	Delivered Member Guide, Staff & Admin Guide, and Usability Testing report under docs/Extras/UserManual/ and docs/Extras/UsabilityTesting/ (Section 2).

SRS closing note. The Software Requirements Specification remains the behavioural baseline cited by the MIS/MG and VnV artifacts. Rev 1 verification (unit cases UT-M*, FR/NFR matrices, and scope honesty in the VnV Report) traces execution and documentation back to that baseline; Table 3 and the SRS row of Table 6 document how Rev 1 SRS edits respond to feedback alongside the HA alignment described above.

5 Economic Considerations (LO23)

MacSync is not primarily a retail product: it is a society-owned event platform for the McMaster Engineering Society (MES), funded through student fees and club operations rather than through direct app sales. So the money story is less about selling copies of the app and more about whether MES gets value and what it costs to keep the system running.

Is there a “market”? There is clear demand for the problem MacSync solves. MES and affiliated clubs already run large events with hundreds of attendees; today they stitch together forms, spreadsheets, social posts, and ad hoc payments. A centralized registration, ticketing, seating, notification, and check-in system reduces volunteer labour, cuts error and dispute risk around payments and attendance, and gives students a single trustworthy place to act. Comparable needs exist at other student societies, but our first “market” is MES-run programming at McMaster.

Marketing / adoption. For us, “marketing” really meant getting people to trust and use the app, not big ad campaigns. Concretely, that includes: MES endorsing the app as the official channel for listed events; club executives being trained on the admin portal; listings in the MacSync mobile app on the Apple App Store and Google Play Store; orientation-adjacent comms (e.g. society

newsletters, Discord/Instagram pointers); and optional demo sessions for organizers. Success is measured by active registrations and repeat use per event, not by download counts alone.

Cost to produce a deployable version. A rough order-of-magnitude for a version suitable for real MES events (excluding sunk student labour) includes: cloud hosting for APIs and web admin (tens to low hundreds of CAD per month depending on traffic and redundancy choices); domains and certificates; Apple Developer and Google Play programme fees for store distribution; Stripe transaction fees on ticket sales (percentage-based, passed through or absorbed by policy); and admin time for society volunteers (watching for errors, refunds, role changes). If we priced our build time at industry salaries, the project would look expensive; as a capstone, student hours stand in for a lot of that cash cost.

What would we charge? We would not charge a separate fee for downloading MacSync. Revenue belongs to ticketed events: prices are set per event, with Stripe handling card presentment. Any “price” for the platform itself would be an internal MES line item (e.g. hosting subsidy from the society), not something students pay for separately from tickets.

Units to break even. A classic break-even on app unit sales does not apply. Break-even is better framed as: does event margin after payment processing cover ongoing hosting and support? For a volunteer-run society, even a small surplus on major formals, combined with avoidance of manual reconciliation time, can justify fixed operating costs—provided incidents (failed payments, refund disputes) stay manageable.

Attracting users if not “sold.” The practical approach is to tie the app to events people already want: formals, pub nights, and similar. If those events use MacSync for sign-up, students do not have to “discover” the app by accident. Good guides (our extras), notifications that actually help, and quick check-in at the door make it more likely someone will use the app again next time.

How many potential users exist? McMaster Engineering enrolls on the order of thousands of undergraduate students; any of them may attend MES-facing events. Typical large MES events draw hundreds of attendees. Organizer-side users are smaller: tens to a few hundred volunteers and executives across clubs with staff roles in the admin portal. That gives a realistic picture of how many people might use MacSync for a first deployment at McMaster, without pretending we are launching province-wide on day one.

6 Reflection on Project Management (LO24)

We ran the project with a few habits that worked for us: at least one full team meeting every week (and more when a deadline was close), and written updates most other days—Discord messages, comments on GitHub issues and PRs, and quick notes so people could read what changed when they had time. We still

used agendas and meeting minutes for the weekly call. GitHub kept the work organized: issues and the board showed who was doing what, and pull requests plus Actions helped catch broken builds before they hit `main`. Live meetings were for bigger decisions and anything that was faster to sort out by talking; everything else did not need another call. The rest of this section compares what we did to the Development Plan, what worked, what did not, and what we would change next time.

6.1 How Does Your Project Management Compare to Your Development Plan

Meetings and communication (largely followed). The Development Plan originally described two standing team meetings per week; in practice we standardized on at least one scheduled team meeting every week, with additional syncs added ad hoc before major deadlines. Between those calls, coordination was mostly in writing: Discord for quick questions, GitHub for anything we wanted a record of on issues and PRs, and shared docs when we were editing rubrics or specs together. That way we still used agendas and meeting minutes from the chair, but nobody had to be online twice a week unless they wanted to. Between Rev 0 and the final deliverables we held seven internal meetings in the documented contribution window, with full member attendance (occasionally with late arrival or early departure). The plan’s 12-hour response expectation was an aspiration for text and review turnaround: it steered urgency on blockers but was not perfectly met during exam and course-crunch periods.

Roles and workflow (followed in spirit with evolution). We kept the named coordinating roles (leader, organizer, reviewer, notetaker/chair, troubleshooter) and the technical ownership split (web primary/secondary, mobile, backend, database, CI/CD). Issues were pre-created for milestones then assigned in meetings so load balanced without relying on ad hoc self-claiming alone. The workflow we documented—feature/`dev/main` branching, small PRs, at least one review, stronger review for auth/payments/schema, and a Definition of Done—matched how we aimed to run the repo.

Technology stack (aligned). We delivered on the planned ecosystem: React Native / Expo for the member app, Next.js for the admin dashboard, service/backend APIs with persisted data, Stripe for payments, and GitHub Actions for CI-style checks on integration paths. The plan’s intent to stage via `dev` and release from `main` matches our operating model even when course deadlines forced pragmatic shortcuts on “production” versus “demo” semantics.

Where we deviated from the written plan. The Development Plan assumed weekly supervisor touchpoints during the tutorial slot; after the strategic pivot to a standalone MacSync (see Section 3), we consolidated supervisor engagement and held one focused stakeholder meeting in the Rev 0-to-final window to align on approach rather than weekly standing check-ins. We also missed the TA document discussion session entirely (scheduling conflicts, religious observance,

and injury among members—as recorded in our final team contribution notes). Only one of five members attended the single scheduled lecture in that period; the rest relied on a teammate summary—workable but not ideal for equal exposure to instructor messaging. These problems came down to scheduling and life outside the project, not to the team skipping our own meetings on purpose.

6.2 What Went Well?

- **Seeing what was actually in progress.** GitHub issues, milestones, and the Kanban columns (**Ready** → **In Progress** → **In Review** → **Done**) made it obvious when we were taking on too much—especially when course docs and VnV sat beside coding work.
- **A steady weekly rhythm.** Meeting once a week (sometimes more), plus short written updates on issues and PRs, meant problems did not sit hidden until the night before a deadline. We could move tasks around when someone got stuck.
- **Work spread across the team.** Commits and issue assignments stayed spread out enough that one person was never the only one who knew how the whole repo worked.
- **Pull request reviews.** We tried to review before merging, and to look harder at anything touching login, payments, or the database. That caught mistakes earlier than if we had only tested right before a demo.
- **Team charter expectations.** The charter asked for attendance, steady GitHub activity, and hitting dates we set in meetings. Those rules felt fair: when someone slipped, they said so early, and we still shipped on time.

6.3 What Went Wrong?

- **We missed the TA document session.** None of us made it to the scheduled TA discussion, so we lost easy face-to-face feedback on drafts. We still went through rubrics and comments on our own, but skipping that meeting was a real miss.
- **Fewer supervisor check-ins after we changed direction.** The Development Plan assumed more regular contact with our supervisor; after we pivoted to a standalone MacSync we only held one focused meeting in the Rev 0-to-final window. That saved time but also meant less ongoing feedback from someone outside the team.
- **Only one person went to lecture.** Everyone else depended on a summary. If that one person missed a detail about a rubric change, the whole team could have been wrong together.

- **One person opened almost all the issues.** It kept labels and titles consistent, but when that person was busy, new work sometimes sat longer before it showed up on the board—we fixed some of that in meetings, but not through a second “issue opener” by default.

6.4 What Would you Do Differently Next Time?

- **Book TA and tutorial time early.** Put TA sessions and important lectures on the calendar as soon as dates are known, and take turns being the person who goes in person so everyone gets a written summary the same day.
- **Short, regular supervisor check-ins after a big change.** After a pivot, a 15–20 minute chat every couple of weeks would have kept our supervisor in the loop without needing a long meeting every time.
- **Share who opens issues.** Use the same templates, but rotate who creates issues by milestone or area so the board never waits on one person.
- **Leave real time for docs and testing.** Course reports and VnV take the same people as the app; next time we would plan that overlap up front instead of assuming we could fit it in at the end.

7 Ethics and Equity (LO18)

Equity and universal design showed up in how we thought about who was stressed when things went wrong, not only about abstract rules. Student attendees mostly worry about wasting money, missing a seat on a bus, or standing in a long line without a clear ticket—so we optimized for clear steps, obvious errors with next actions, and tickets that work on a phone. MES organizers worry about fairness between clubs, not overselling capacity, and handling refunds without a spreadsheet war—so we invested in role limits, audit-friendly flows in the admin tool, and documentation that does not assume everyone is a developer. Thinking from those angles is how we tried to practice empathy: we listened to usability testers describe where they got stuck and read TA feedback about labeling and validation the same way an organizer or first-year student might experience it.

Concrete steps toward equitable treatment. We tied sensitive attendee data to permissions: staff see what they need for an event, not the whole campus roster by default. The problem statement and hazard analysis already flagged dietary and accessibility-related information and privacy; design and verification work treated those categories as data that deserves protection, not as optional notes. We aimed for a mobile-first path for members because many students live on their phones and may not have a laptop at the venue. User-facing manuals spelled out verification and refund policy limitations (for example, who to contact if a refund stalls) so members are not left guessing. Usability

testing deliberately included two kinds of stakeholders—attendee-style tasks and organizer-style tasks—so we did not only optimize for our own team.

Universal design (what we actually built toward). Examples include: clear navigation patterns in the mobile app (fixed tabs, consistent language between docs and UI where we could); notification content that works both as a push banner and inside the app for people who mute pushes; large touch targets and simple flows in testing scenarios (buy ticket, find QR, request refund); and honest error feedback called out in usability findings (replacing vague “something went wrong” messages was flagged as important because it helps everyone, not only expert users). Language in the Member and Admin guides put warnings next to destructive actions (delete user, delete event) so consequences are visible before someone clicks.

Where we fell short (honest misses). MacSync still assumes a valid `@mcmaster.ca` account for ordinary registration. That matches MES’s world most of the time, but it is not equitable for guests, partners, or students who do not use that email pattern without a manual workaround. We also did not complete a full, formal WCAG-style accessibility audit with screen-reader users in our capstone window; usability covered a small sample and some checkpoints (contrast, clarity) but is not the same as certified compliance. The problem statement mentioned richer dietary/accessibility matching work than we fully automated; parts of that vision stayed at documentation or process level rather than a polished product feature. Finally, heavy reliance on smartphones helps many users but can exclude someone without a capable device unless staff fall back to manual check-in—a gap to close with explicit organizer playbooks, not only software.

Looking forward. If MacSync continues past the course, the next fairness wins would be: documented offline/edge-case check-in procedures, expanded assistive-technology testing, clearer pathways for guests and others without a McMaster email if policy allows, and measuring whether all major flows remain usable after UI churn. Naming those gaps is part of the same ethical duty as listing what we improved.

8 Reflection on Capstone

This capstone pulled together work we had only ever done in pieces during earlier terms. The connections we sketched in our SRS-phase reflection still ring true at the end of the project: the same courses kept paying off as we moved from requirements documents to a real mobile app, admin web app, backend, and verification package.

8.1 Which Courses Were Relevant

- **3RA3 (Software Requirements).** This course was the direct preparation for structuring requirements, naming stakeholders, and writing an SRS-style specification. It also introduced us to hazard thinking and the

idea that verification should trace back to requirements—skills we reused in the Hazard Analysis, MIS-linked unit tests, and VnV artifacts instead of treating those documents as one-off essays.

- **3A04 (Software Design).** Design coursework helped us think in terms of modules, boundaries, and interfaces rather than a single giant program. That mindset matched how we separated attendee flows, admin flows, payments, and access control, and it made the Module Guide and MIS feel like natural extensions of how we already thought about the system.
- **2AA4 (Introduction to Software Development).** 2AA4 was our first serious experience building a multi-milestone group project: splitting features, iterating, and stacking each demo on the last. Capstone amplified that pattern (documentation sprints beside coding sprints), but the basic habit—agree on a thin vertical slice, then widen it—came from that course.
- **3S03 (Software Testing).** Testing coursework gave us a shared vocabulary for unit versus integration concerns, defect thinking, and why automated checks belong on pull requests. We leaned on those ideas when we argued for review gates and when we filled out implementation verification in the VnV Plan.
- **4HC3 (Human-Centred Interface Design).** HCI work pushed us to justify choices from a user and stakeholder perspective, not only from what is easiest to implement. That showed up in usability task design, in how we interpreted TA and tester feedback, and in the extras (Member and Admin guides written for non-developers).

8.2 Knowledge/Skills Outside of Courses

Even with those courses, we still had to learn on the job in a few areas that the degree touches only lightly or not at all in one place:

- **Stripe and payment flows.** Courses discussed security at a high level; integrating a live payment provider, webhooks, refunds, and sandbox testing required reading vendor docs and small experiments we owned ourselves.
- **React Native / Expo and modern front-end tooling.** We knew React ideas from design and web work, but mobile builds, store constraints, and Expo-specific debugging were mostly new in depth.
- **Production-shaped GitHub practice.** Branch protection, labels, project boards, and review etiquette at capstone scale went beyond what a single assignment usually enforces; we learned that by running the repo like a small team, not a homework fork.
- **Operations and deployment vocabulary.** Choosing hosting paths, environment secrets, and “what breaks on demo day” troubleshooting drew on tutorials and peers more than on any one lecture block.

- **Professional writing under rubrics.** CEAB-style reflection, trace tables, and instructor templates are a genre of their own; we got faster by reading past examples, TA comments, and each other's drafts.

In short, courses gave the skeleton (requirements, architecture, testing mindset, user focus), and capstone forced us to put muscle on it with stack-specific tools, integration risk, and documentation discipline we will reuse in internships or future projects.